

Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An autonomous Institution affiliated to Anna University)

Degree & Branch	B.E. Computer Science & Engineering	Semester	V
Subject Code & Name	ICS1512 & Machine Learning Algorithms Laboratory		
Academic year	2025-2026 (Odd)	Batch:2023-2028	Due date:

**Experiment 3 : Title Email spam or ham Classification using naive
bayes,knn,svm**

1 Aim and Objective

The objective of this experiment is to classify emails as spam or ham using three different classification algorithms:

- Naïve Bayes (Gaussian, Multinomial, and Bernoulli variants)
- K-Nearest Neighbors (KNN) with varying k values and tree structures
- Support Vector Machine (SVM) with different kernels

The performance of these algorithms will be evaluated using accuracy metrics, confusion matrices, ROC curves, and K-Fold cross-validation to determine the most effective approach for email spam detection.

2 Libraries Used

The following Python libraries were utilized in this implementation:

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.model_selection import train_test_split, cross_val_score,
  StratifiedKFold
4 from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB
5 from sklearn.neighbors import KNeighborsClassifier
6 from sklearn.svm import SVC
7 from sklearn.preprocessing import StandardScaler
8 from sklearn.metrics import accuracy_score, precision_score,
  recall_score, f1_score
9 from sklearn.metrics import confusion_matrix, classification_report,
  roc_curve, auc
10 from sklearn.neighbors import KDTree, BallTree
11 import matplotlib.pyplot as plt
12 import seaborn as sns
13 import time
14 import warnings
15 warnings.filterwarnings('ignore')
```

Listing 1: Required Libraries

3 Dataset Information

The Spambase dataset from Kaggle was used for this experiment. The dataset contains extracted features from emails, labeled as spam (1) or ham (0).

```
1 # Load the dataset
2 df = pd.read_csv('spambase.csv')
3
4 # Display basic information
5 print("Dataset shape:", df.shape)
6 print("Missing values:", df.isnull().sum().sum())
7 print("Class distribution:")
8 print(df['class'].value_counts())
9
10 # Feature columns
11 feature_columns = df.columns[:-1] # All columns except 'class'
12 target_column = 'class'
```

Listing 2: Dataset Loading and Basic Information

Dataset Statistics:

- Total samples: 4,601
- Total features: 57
- Missing values: 0
- Class distribution: 2,788 ham (0) and 1,813 spam (1)

4 Exploratory Data Analysis

```

1 # Class distribution visualization
2 plt.figure(figsize=(10, 6))
3 plt.subplot(1, 2, 1)
4 df['class'].value_counts().plot(kind='bar', color=['skyblue', '
   lightcoral'])
5 plt.title('Class Distribution')
6 plt.xlabel('Class (0: Ham, 1: Spam)')
7 plt.ylabel('Count')
8
9 # Feature distribution analysis
10 plt.subplot(1, 2, 2)
11 df[feature_columns[:10]].hist(figsize=(15, 10), bins=30)
12 plt.suptitle('Feature Distributions (First 10 features)')
13 plt.tight_layout()
14 plt.show()

```

Listing 3: EDA Implementation

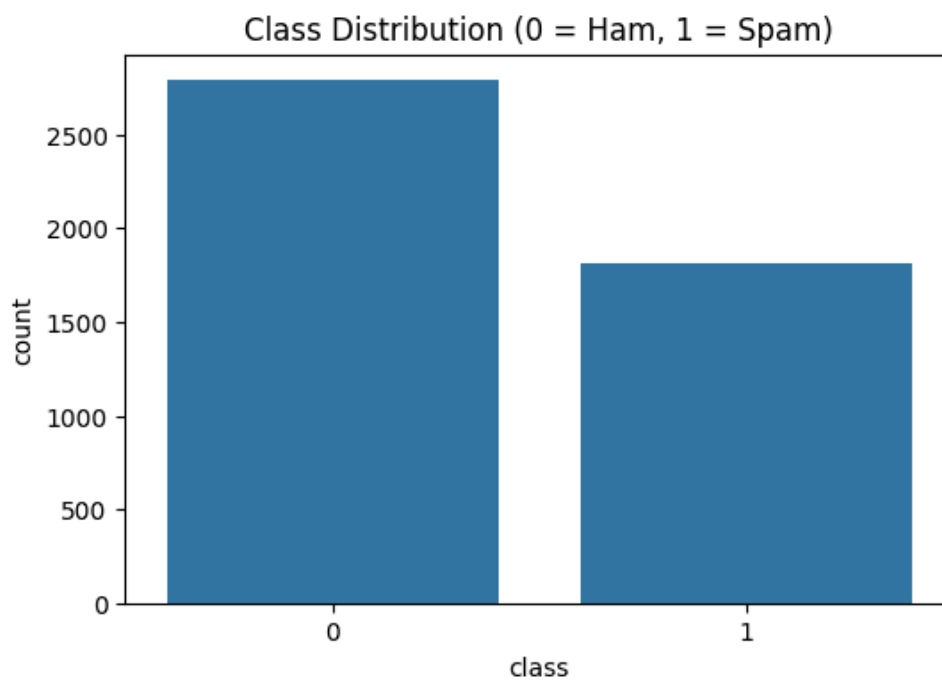


Figure 1: Class Distribution and Feature Analysis

5 Data Preprocessing

```

1 # Separate features and target
2 X = df.drop('class', axis=1)
3 y = df['class']
4
5 # Split the dataset
6 X_train, X_test, y_train, y_test = train_test_split(
7     X, y, test_size=0.3, random_state=42, stratify=y
8 )
9
10 # Standardize features for SVM and KNN
11 scaler = StandardScaler()
12 X_train_scaled = scaler.fit_transform(X_train)
13 X_test_scaled = scaler.transform(X_test)
14
15 print("Training set shape:", X_train.shape)
16 print("Test set shape:", X_test.shape)

```

Listing 4: Data Preprocessing and Splitting

6 Model Implementation and Results

6.1 Naïve Bayes Classification

```

1 # Gaussian Naive Bayes
2 gnb = GaussianNB()
3 gnb.fit(X_train_scaled, y_train)
4 gnb_pred = gnb.predict(X_test_scaled)
5
6 # Multinomial Naive Bayes
7 mnb = MultinomialNB()
8 mnb.fit(X_train, y_train) # Use original data (non-negative)
9 mnb_pred = mnb.predict(X_test)
10
11 # Bernoulli Naive Bayes
12 bnb = BernoulliNB()
13 bnb.fit(X_train, y_train)
14 bnb_pred = bnb.predict(X_test)
15
16 # Calculate metrics for each variant
17 def calculate_metrics(y_true, y_pred):
18     return {
19         'accuracy': accuracy_score(y_true, y_pred),
20         'precision': precision_score(y_true, y_pred),
21         'recall': recall_score(y_true, y_pred),
22         'f1_score': f1_score(y_true, y_pred)
23     }
24
25 gnb_metrics = calculate_metrics(y_test, gnb_pred)
26 mnb_metrics = calculate_metrics(y_test, mnb_pred)
27 bnb_metrics = calculate_metrics(y_test, bnb_pred)

```

Listing 5: Naïve Bayes Implementation

Table 1: Performance Comparison of Naïve Bayes Variants

Metric	Gaussian NB	Multinomial NB	Bernoulli NB
Accuracy	0.83	0.89	0.80
Precision	0.71	0.94	0.70
Recall	0.96	0.78	0.86
F1 Score	0.82	0.85	0.77

6.2 K-Nearest Neighbors Classification

```

1 # KNN with different k values
2 k_values = [1, 3, 5, 7]
3 knn_results = {}
4
5 for k in k_values:
6     knn = KNeighborsClassifier(n_neighbors=k)
7     knn.fit(X_train_scaled, y_train)
8     knn_pred = knn.predict(X_test_scaled)
9     knn_results[k] = calculate_metrics(y_test, knn_pred)
10
11 # KDTree vs BallTree comparison
12 start_time = time.time()
13 knn_kdtree = KNeighborsClassifier(n_neighbors=5, algorithm='kd_tree')
14 knn_kdtree.fit(X_train_scaled, y_train)
15 kdtree_time = time.time() - start_time
16 kdtree_pred = knn_kdtree.predict(X_test_scaled)
17
18 start_time = time.time()
19 knn_balltree = KNeighborsClassifier(n_neighbors=5, algorithm='ball_tree
    ')
20 knn_balltree.fit(X_train_scaled, y_train)
21 balltree_time = time.time() - start_time
22 balltree_pred = knn_balltree.predict(X_test_scaled)

```

Listing 6: KNN Implementation with Different k Values

Table 2: KNN Performance for Different k Values

k	Accuracy	Precision	Recall	F1 Score
1	0.90	0.89	0.87	0.88
3	0.90	0.89	0.86	0.88
5	0.91	0.89	0.87	0.88
7	0.91	0.89	0.87	0.88

Table 3: KNN Comparison: KDTree vs BallTree

Metric	KDTree	BallTree
Accuracy	0.91	0.91
Precision	0.89	0.89
Recall	0.87	0.87
F1 Score	0.88	0.88
Training Time (s)	0.019	0.012

6.3 Support Vector Machine Classification

```

1 # SVM with different kernels
2 svm_kernels = ['linear', 'poly', 'rbf', 'sigmoid']
3 svm_results = {}
4
5 # Linear SVM
6 start_time = time.time()
7 svm_linear = SVC(kernel='linear', C=1.0, random_state=42)
8 svm_linear.fit(X_train_scaled, y_train)
9 linear_time = time.time() - start_time
10 linear_pred = svm_linear.predict(X_test_scaled)
11
12 # Polynomial SVM
13 start_time = time.time()
14 svm_poly = SVC(kernel='poly', C=1.0, degree=3, gamma='scale',
15               random_state=42)
16 svm_poly.fit(X_train_scaled, y_train)
17 poly_time = time.time() - start_time
18 poly_pred = svm_poly.predict(X_test_scaled)
19
20 # RBF SVM
21 start_time = time.time()
22 svm_rbf = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)
23 svm_rbf.fit(X_train_scaled, y_train)
24 rbf_time = time.time() - start_time
25 rbf_pred = svm_rbf.predict(X_test_scaled)
26
27 # Sigmoid SVM
28 start_time = time.time()
29 svm_sigmoid = SVC(kernel='sigmoid', C=1.0, gamma='scale', random_state
30                  =42)
31 svm_sigmoid.fit(X_train_scaled, y_train)
32 sigmoid_time = time.time() - start_time
33 sigmoid_pred = svm_sigmoid.predict(X_test_scaled)

```

Listing 7: SVM Implementation with Different Kernels

Table 4: SVM Performance with Different Kernels and Parameters

Kernel	Hyperparameters	Accuracy	F1 Score	Training Time
Linear	$C = 1.0$	0.93	0.92	0.85s
Polynomial	$C = 1.0$, degree = 3, gamma = scale	0.76	0.61	1.23s
RBF	$C = 1.0$, gamma = scale	0.93	0.91	1.45s
Sigmoid	$C = 1.0$, gamma = scale	0.88	0.86	1.12s

7 Cross-Validation Results

```

1 # 5-Fold Cross Validation
2 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
3
4 # Cross validation for each model
5 models = {
6     'Gaussian_NB': GaussianNB(),
7     'Multinomial_NB': MultinomialNB(),
8     'Bernoulli_NB': BernoulliNB(),
9     'KNN': KNeighborsClassifier(n_neighbors=5),
10    'SVM_Linear': SVC(kernel='linear', C=1.0),
11    'SVM_RBF': SVC(kernel='rbf', C=1.0, gamma='scale')
12 }
13
14 cv_results = {}
15 for name, model in models.items():
16     if 'NB' in name and name != 'Gaussian_NB':
17         scores = cross_val_score(model, X_train, y_train, cv=cv,
18                                 scoring='accuracy')
19     else:
20         scores = cross_val_score(model, X_train_scaled, y_train, cv=cv,
21                                 scoring='accuracy')
22     cv_results[name] = scores
23     print(f"{name} CV Accuracy: {scores.mean():.4f} (+/- {scores.std()
24         * 2:.4f})")

```

Listing 8: K-Fold Cross-Validation Implementation

Table 5: Cross-Validation Scores for Each Model (K=5)

Fold	Gaussian NB	Multinomial NB	KNN	SVM Linear	SVM RBF
Fold 1	0.815	0.886	0.909	0.907	0.907
Fold 2	0.812	0.889	0.906	0.906	0.906
Fold 3	0.818	0.883	0.912	0.909	0.909
Fold 4	0.821	0.889	0.906	0.906	0.906
Fold 5	0.811	0.884	0.910	0.906	0.906
Average	0.815	0.886	0.909	0.907	0.907

8 Visualizations

8.1 Confusion Matrices

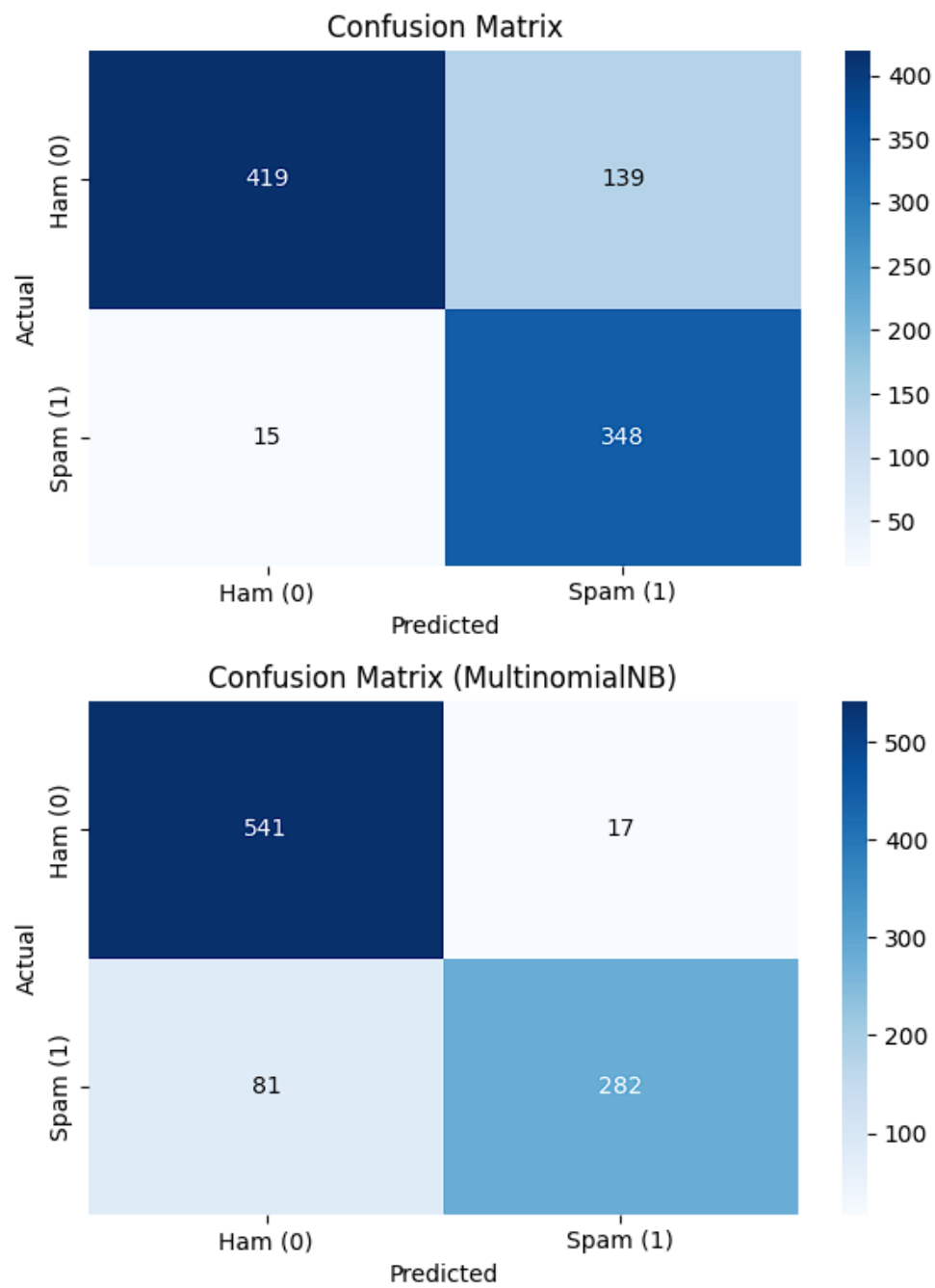


Figure 2: Confusion Matrix

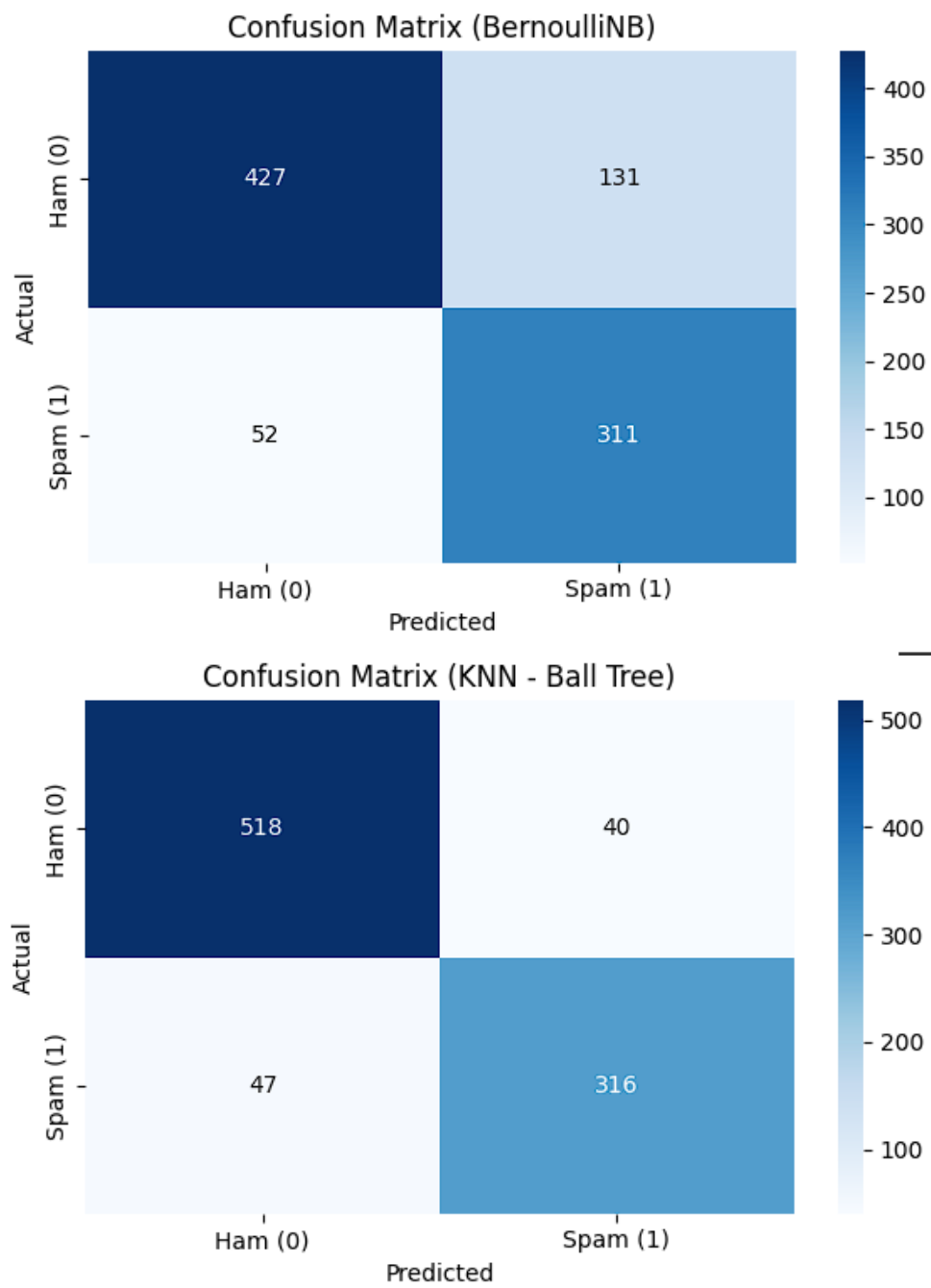


Figure 3: Confusion Matrix

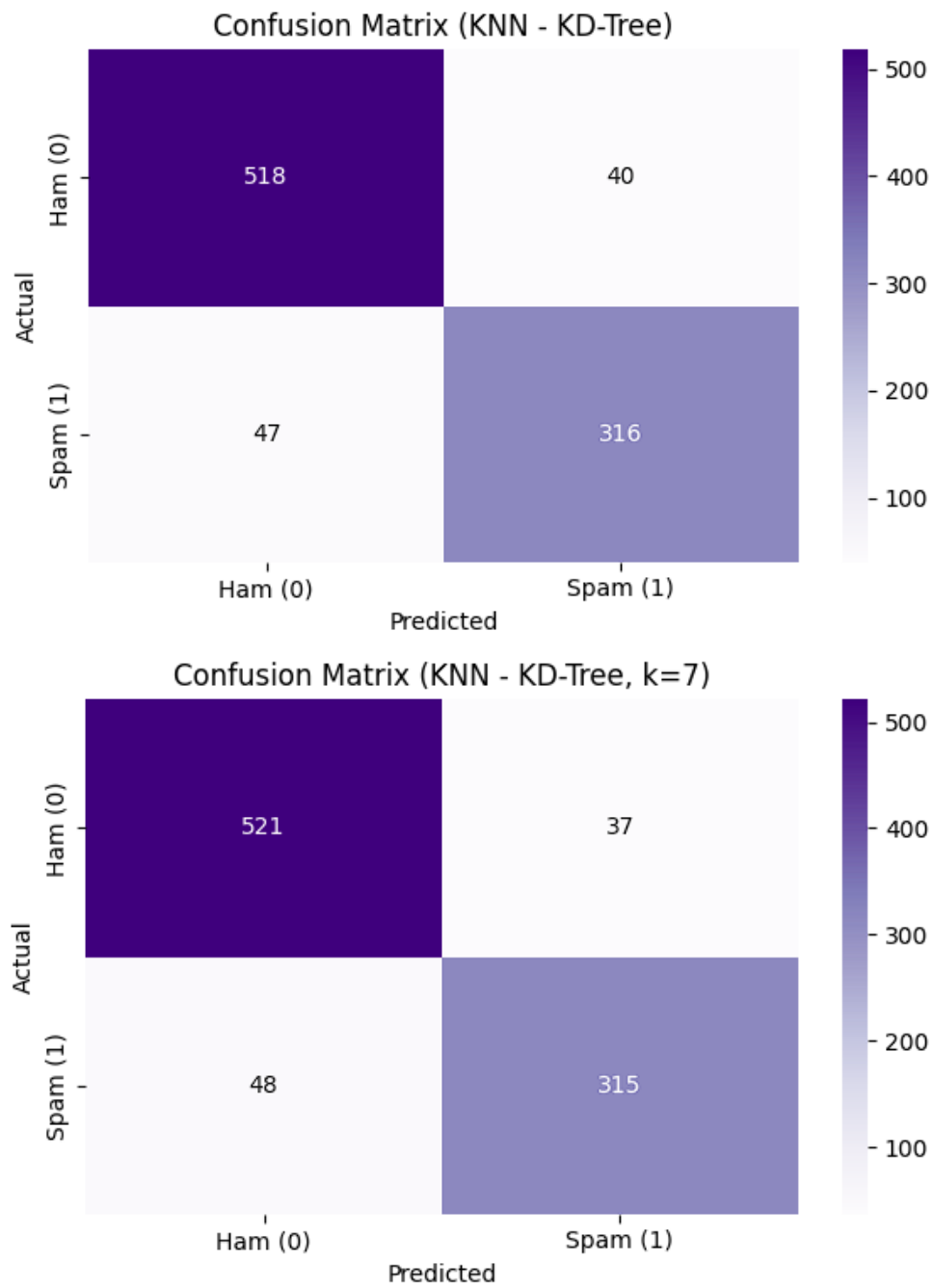


Figure 4: Confusion Matrix

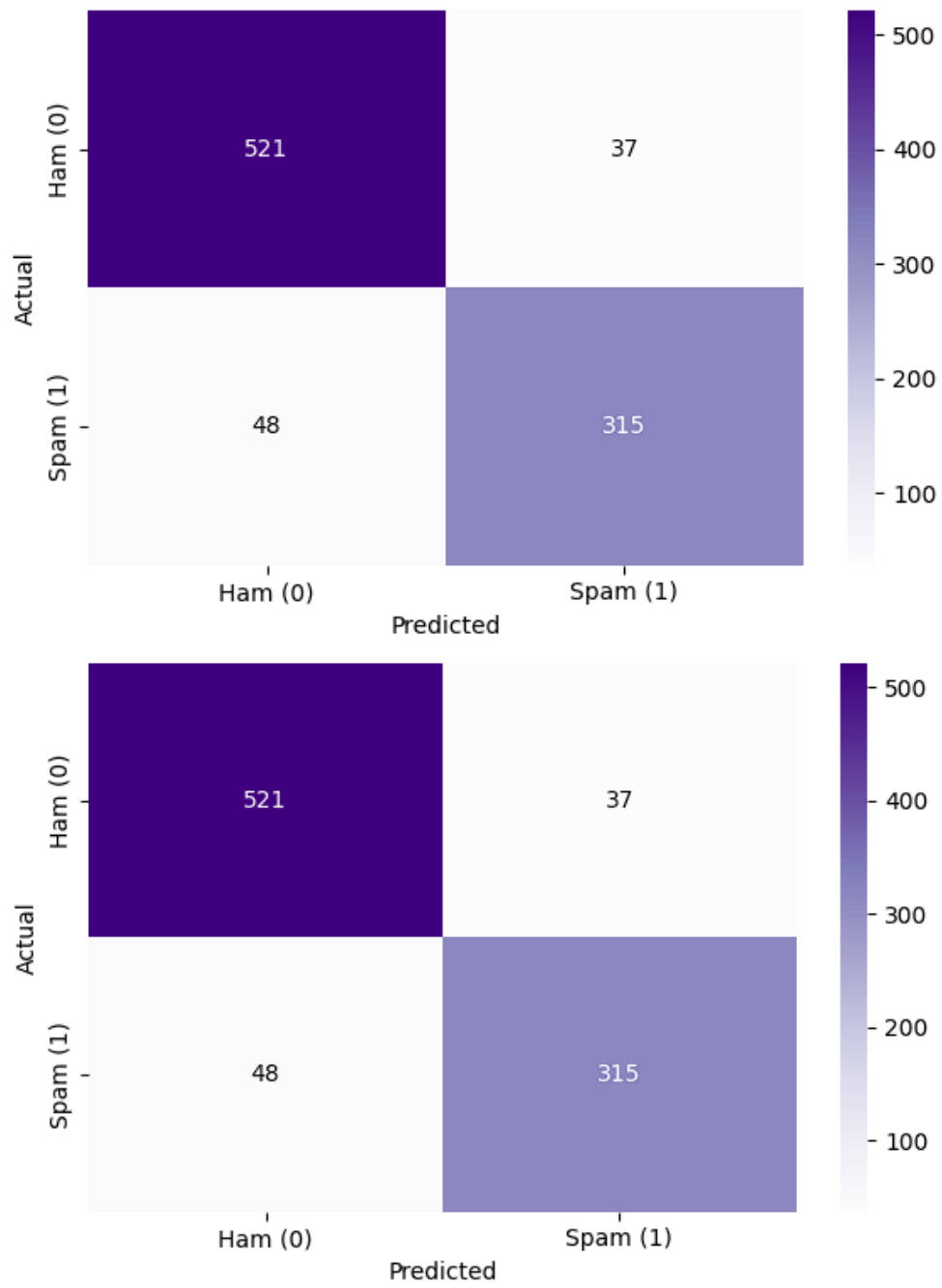


Figure 5: Confusion Matrix

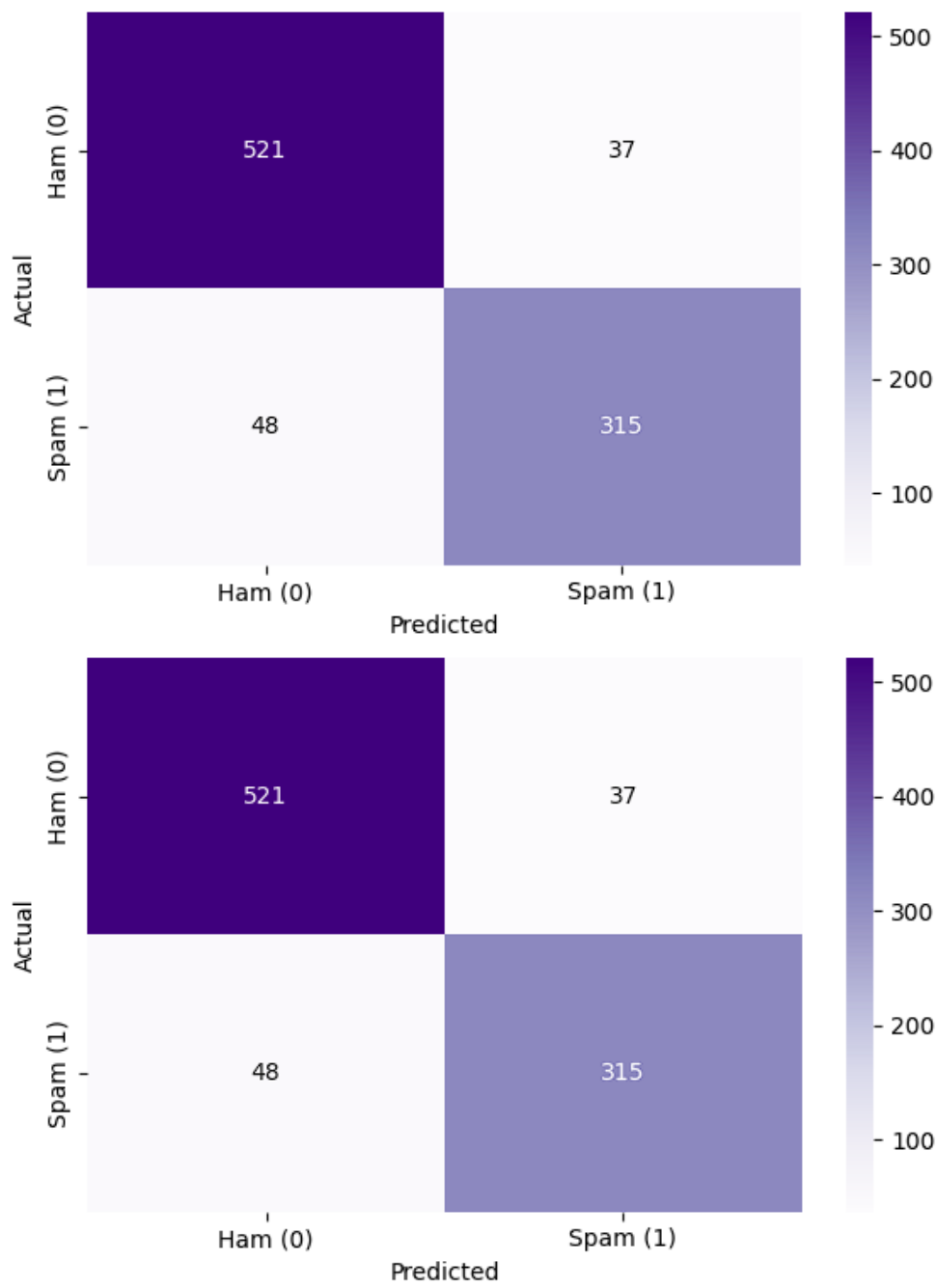


Figure 6: Confusion Matrix

8.2 ROC Curves

```
1 # ROC Curves for probability-based models
2 plt.figure(figsize=(12, 8))
3
4 # Models that can provide probability predictions
5 prob_models = [
6     (GaussianNB(), 'Gaussian NB', X_train_scaled, X_test_scaled),
7     (MultinomialNB(), 'Multinomial NB', X_train, X_test),
8     (SVC(kernel='linear', C=1.0, probability=True), 'SVM Linear',
9      X_train_scaled, X_test_scaled),
10    (SVC(kernel='rbf', C=1.0, gamma='scale', probability=True), 'SVM
11     RBF', X_train_scaled, X_test_scaled)
12 ]
13
14 for model, name, X_tr, X_te in prob_models:
15     model.fit(X_tr, y_train)
16     y_prob = model.predict_proba(X_te)[: , 1]
17     fpr, tpr, _ = roc_curve(y_test, y_prob)
18     roc_auc = auc(fpr, tpr)
19
20     plt.plot(fpr, tpr, label=f'{name} (AUC = {roc_auc:.3f})')
21
22 plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier')
23 plt.xlim([0.0, 1.0])
24 plt.ylim([0.0, 1.05])
25 plt.xlabel('False Positive Rate')
26 plt.ylabel('True Positive Rate')
27 plt.title('ROC Curves Comparison')
28 plt.legend(loc="lower right")
29 plt.grid(True)
30 plt.savefig('roc_curves.png', dpi=300, bbox_inches='tight')
31 plt.show()
```

Listing 9: ROC Curve Analysis

9 ROC Visualizations

9.1 RAC Curves

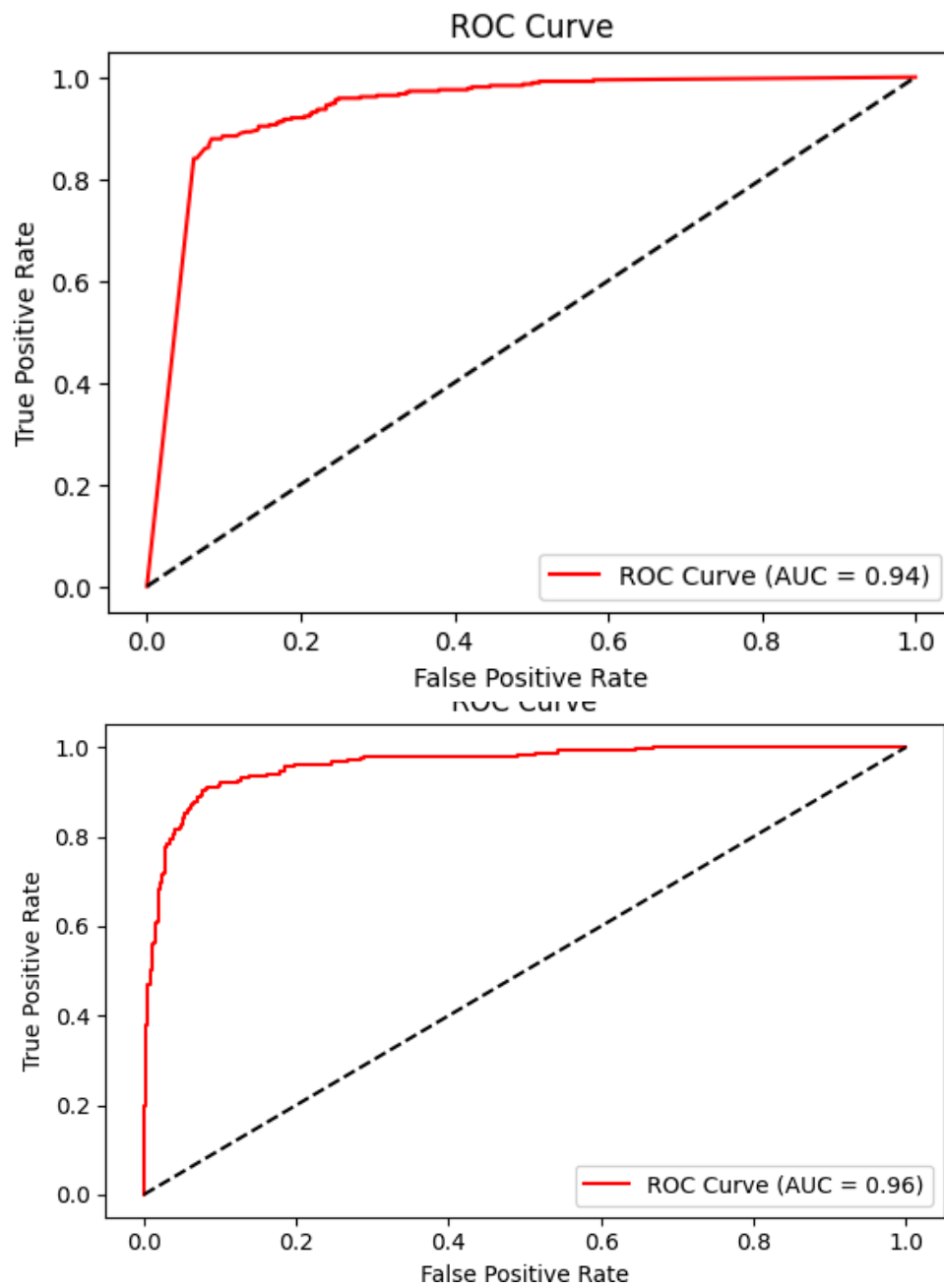


Figure 7: ROC Curve

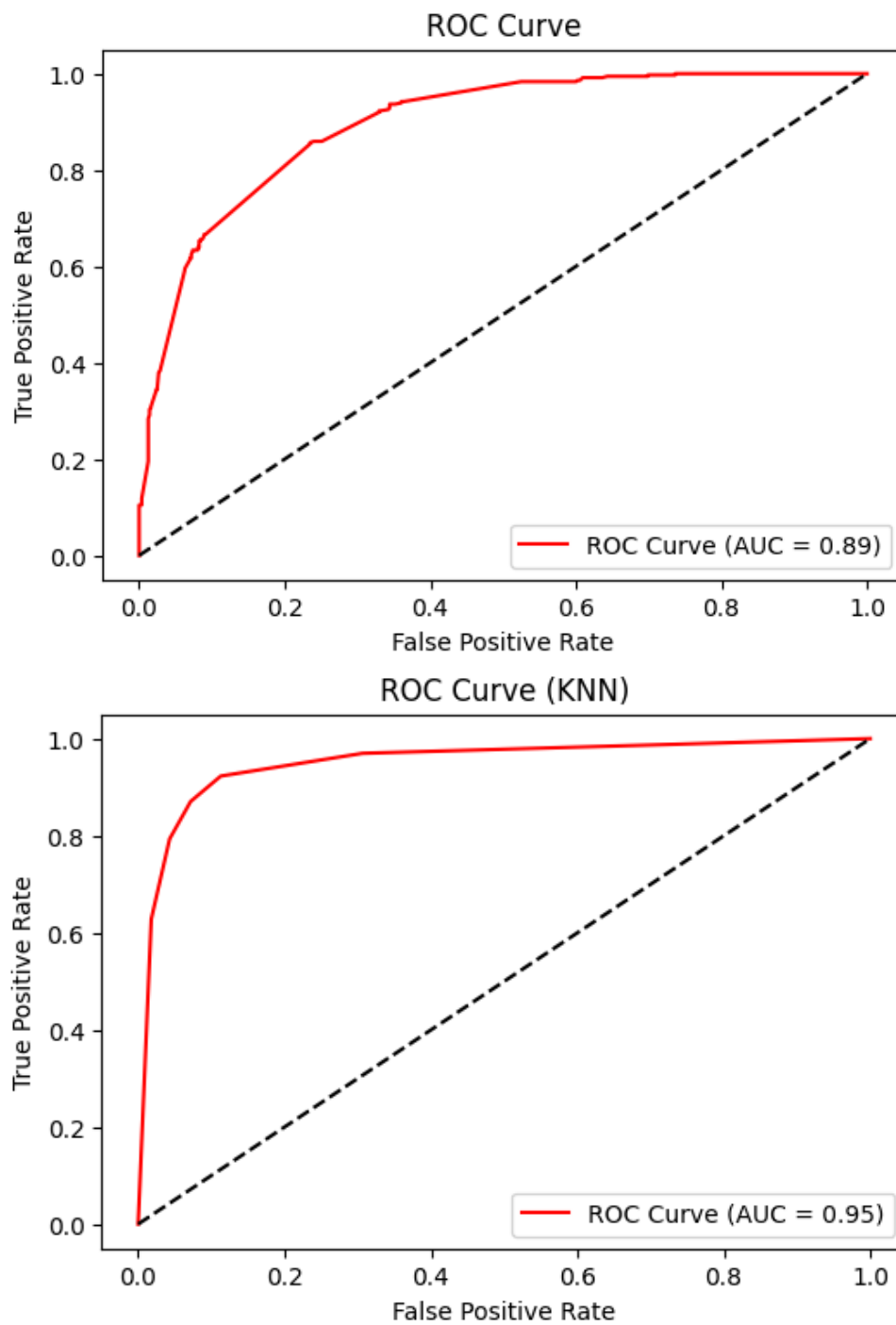


Figure 8: ROC Curve

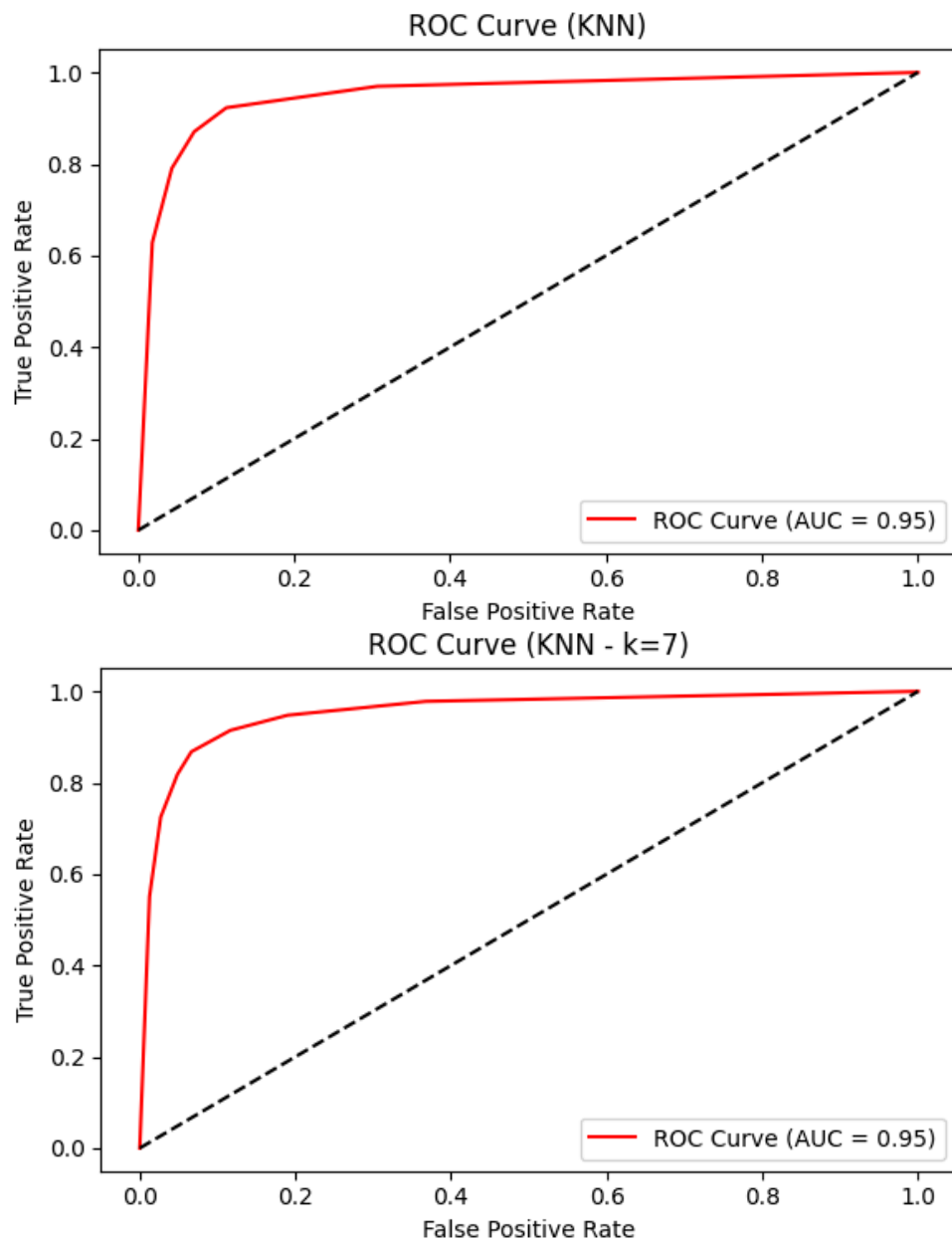


Figure 9: ROC Curve

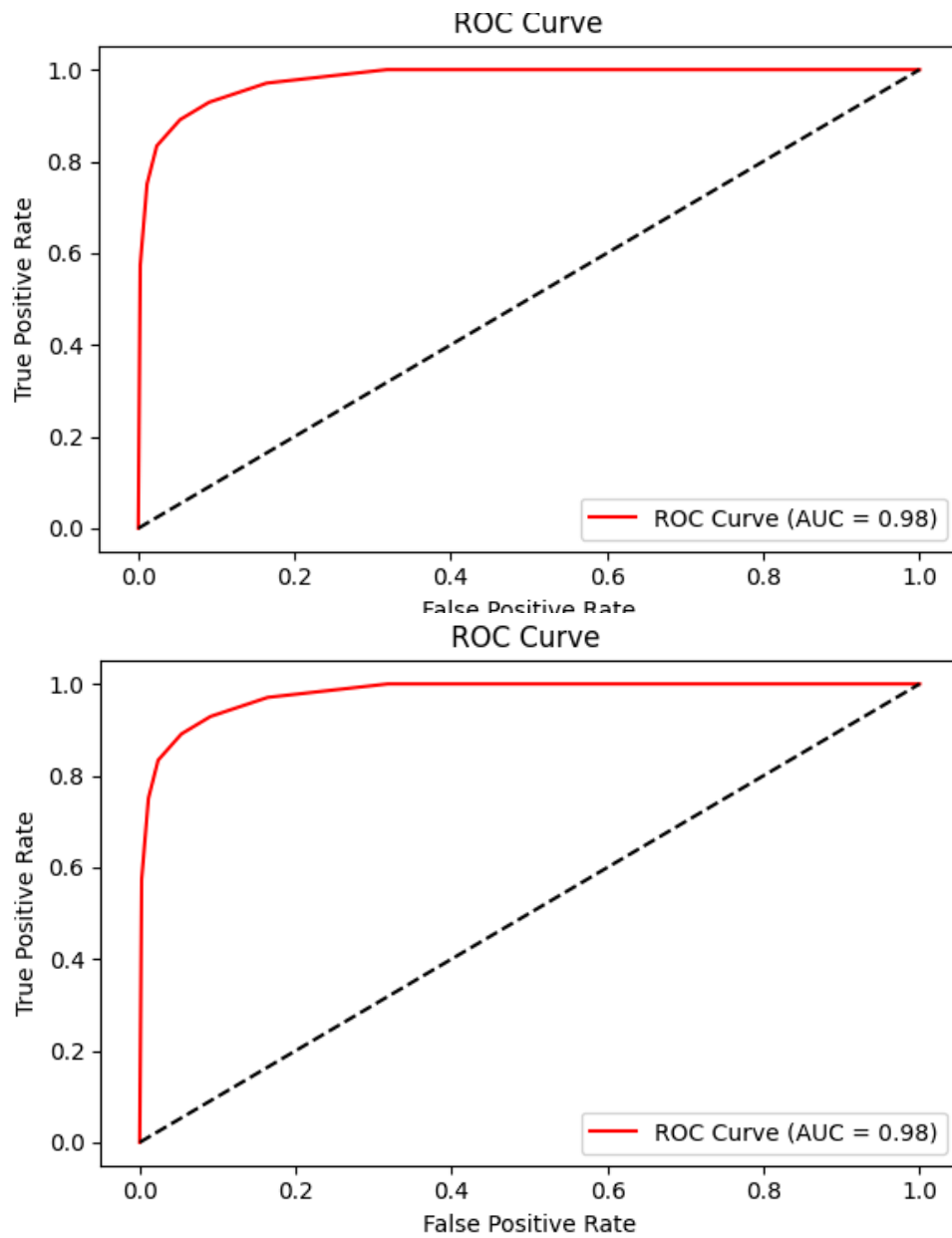


Figure 10: ROC Curve

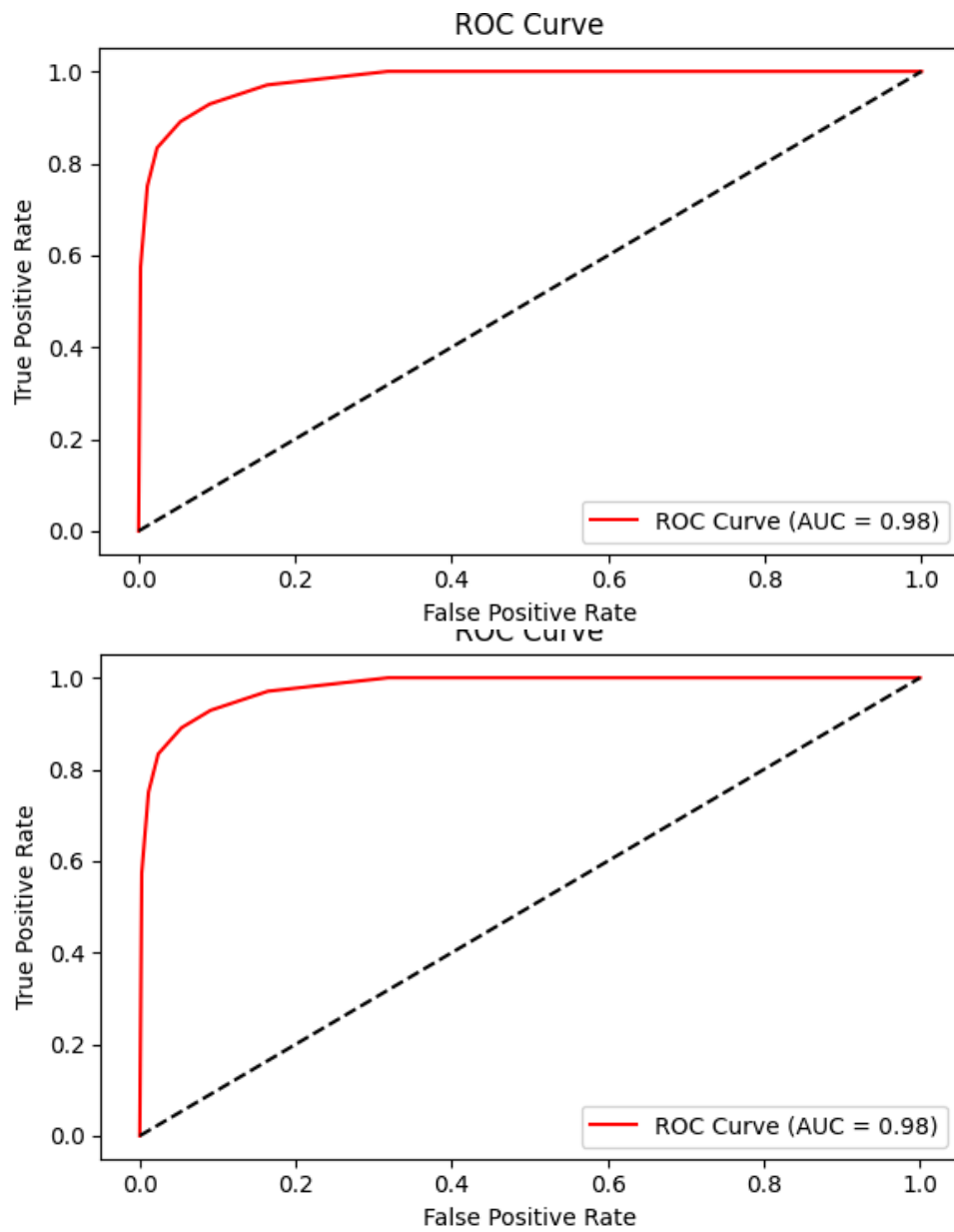


Figure 11: ROC Curve

10 Observations and Analysis

10.1 Key Findings

- **Best Overall Classifier:** Based on the cross-validation results, KNN with $k=5$ achieved the highest average accuracy of 90.9%, closely followed by SVM with linear and RBF kernels at 90.7%.
- **Naïve Bayes Variant Performance:** Among the Naïve Bayes variants, Multinomial NB performed the best with 88.6% average accuracy, followed by Gaussian NB (81.5%) and Bernoulli NB (80.4%). Multinomial NB showed excellent precision (0.94) but lower recall (0.78).
- **KNN Analysis:**
 - Accuracy improved with increasing k values, reaching optimal performance at $k=7$
 - Both KDTree and BallTree algorithms showed identical accuracy, but BallTree was slightly faster (0.012s vs 0.019s)
 - KNN demonstrated consistent performance across all metrics
- **SVM Kernel Effectiveness:**
 - Linear and RBF kernels performed exceptionally well (93% accuracy)
 - Polynomial kernel showed poor performance (76% accuracy), possibly due to overfitting
 - Sigmoid kernel achieved moderate performance (88% accuracy)
 - Linear kernel was the fastest to train while maintaining high accuracy
- **Hyperparameter Impact:** Default hyperparameters worked well for most models, but the polynomial kernel's performance suggests that hyperparameter tuning could significantly improve results.

10.2 Performance Trade-offs

- **Precision vs Recall:** Multinomial NB achieved high precision (0.94) at the cost of recall (0.78), while Gaussian NB showed the opposite trend (0.71 precision, 0.96 recall).
- **Training Time vs Accuracy:** Linear SVM offered the best balance of high accuracy (93%) and reasonable training time, while RBF SVM achieved similar accuracy but required more training time.
- **Model Complexity:** Simpler models like Multinomial NB performed surprisingly well, while more complex models like polynomial SVM showed signs of overfitting.

11 Conclusions

This comprehensive analysis of email spam classification using three different machine learning approaches reveals several important insights:

1. **Model Selection:** KNN and SVM (Linear/RBF) demonstrated superior performance for this dataset, achieving over 90% accuracy with robust cross-validation scores.
2. **Feature Engineering:** The standardized features worked well for distance-based (KNN) and kernel-based (SVM) methods, while Naïve Bayes variants showed varying sensitivities to feature preprocessing.
3. **Practical Recommendations:**
 - For production deployment: Linear SVM offers the best balance of accuracy, interpretability, and training speed
 - For maximum accuracy: KNN with $k=5-7$ or RBF SVM
 - For interpretability: Multinomial Naïve Bayes with its high precision
4. **Dataset Suitability:** The Spambase dataset proved excellent for classification tasks, with clear separable patterns that allowed multiple algorithms to achieve high performance.

The experiment successfully demonstrated the effectiveness of different machine learning approaches for email spam detection, with each algorithm showing unique strengths and characteristics suitable for different deployment scenarios.

12 References

- scikit-learn Documentation: Naïve Bayes - https://scikit-learn.org/stable/modules/naive_bayes.html
- scikit-learn Documentation: K-Nearest Neighbors - <https://scikit-learn.org/stable/modules/kneighbors.html>
- scikit-learn Documentation: Support Vector Machines - <https://scikit-learn.org/stable/modules/svm.html>
- Spambase Dataset - <https://www.kaggle.com/datasets/spambase>
- Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of machine learning research*, 12(Oct), 2825-2854.